

VR-Sets-ZFA

Операциональная референциальная семантика для нефундированных множеств

Виталий Резник

Версия 1.0.0 — 25 мая 2026

Аннотация

VR-Sets-ZFA — фундаментальное расширение VR-Sets, дающее операциональную референциальную семантику для нефундированных множеств в Lean 4. Конструкция берёт коиндуктивный параллель индуктивного PSet из mathlib через PFunctor.M, определяет экстенсиональный квотиент OSetZFA по кобисимуляции, доказывает аксиому антифундированности Ацеля (AFA) как теорему из универсального свойства финальной коалгебры, и устанавливает точное вложение $\text{OSet} \rightarrow \text{OSetZFA}$ фундированных множеств в ZFA-универсум.

Работа формально разрешает Гипотезу IV.2 из VR-Sets: тип, удовлетворяющий AFA, может быть построен в операциональной онтологии VR. Атом Куайна и омега-цепь — машинно-проверенные свидетельства невырожденности; коллапс бисимуляции, отождествляющий двухвершинный циклический граф с самозамкнутой петлёй, доказан как теорема `cycleDecoration_eq_quineAtom`.

Шестая работа цикла VR. Сопутствующая формализация в Lean 4: DOI 10.5281/zenodo.20368268.

* * *

Часть I — Позиция

I.1 Место в цикле VR

VR-Sets-ZFA — шестая работа цикла VR, следующая после VR (Формальная система), VR-Numbers, VR-Sets, VR-Forms и VR-Audit. Первые четыре работы — фундаментальные; пятая — прикладная. VR-Sets-ZFA — фундаментальная: она расширяет операциональную онтологию VR-Sets до допущения нефундированных множеств через новый коиндуктивный тип, сохраняя совместимость с существующей фундированной конструкцией через точное вложение.

Работа не является аудитом. Это не применение существующего аппарата к классической теореме (как VR-Audit-1 для теоремы Хана–Банаха). Она строит новые фундаментальные типы и доказывает теоремы о них. Принцип `wrapping` из VR-Audit здесь неприменим: в

mathlib нет инфраструктуры нефундированных множеств для оборачивания. Работа строит инфраструктуру сама, используя PFuncтор.M из mathlib как субстрат.

I.2 Формальный ответ на Гипотезу IV.2

VR-Sets поставил (в Conjectures.lean) вопрос о том, можно ли построить тип, удовлетворяющий аксиоме антифундированности Ацеля, в операциональной онтологии VR. Гипотеза оставалась открытой, потому что Lean-реализация VR-Sets выбрала индуктивный PSet как основание: PSet.mem_irrefl и PSet.mem_wf следуют из индуктивной структуры, делая ZFA-style нефундированность невозможной в OSet. Граничные свидетели AFA_Refuted и quineAtom_impossible явно зафиксированы в Modes.lean.

VR-Sets-ZFA отвечает на Гипотезу IV.2 конструктивно. Тип OSetZFA = Quotient CoPSet.cobisim, где CoPSet — коиндуктивный параллель PSet через PFuncтор.M, удовлетворяет AFA. Доказательство прямое: AFA_in_OSetZFA устанавливается предъявлением единственной декорации каждого графа через M.corec, с единственностью через экстенциональную кобисимуляцию. Атом Куайна построен конкретно как декорация одновершинного графа с петлёй.

I.3 Чем эта работа не является

VR-Sets-ZFA не заменяет VR-Sets. Существующая фундированная конструкция OSet остаётся без изменений. OSet продолжает быть основанием для VR-Numbers и других работ цикла VR. VR-Sets-ZFA стоит рядом с OSet как расширение, причём OSet точно вкладывается в OSetZFA через embedOSet.

VR-Sets-ZFA не является вкладом в инфраструктуру ZFA для mathlib. В mathlib нет ZFA; эта работа реализована в namespace цикла VR и не предлагает upstream-изменений. Внешние пользователи получают VR-Sets-ZFA, зависимостью от цикла VR, а не импортом из mathlib.

VR-Sets-ZFA не является исчерпывающим изложением теории нефундированных множеств. Она устанавливает AFA и демонстрирует невырожденность, но не формализует алгебру процессов, модальную логику, коалгебраическую семантику или другие приложения AFA. Эти направления остаются как кандидаты для будущей работы.

VR-Sets-ZFA рассматривает только AFA Ацеля. Альтернативные аксиомы антифундированности (SAFA Скотта, BAFA Боффа, FAFA Финслера) не рассматриваются.

* * *

Часть II — Архитектура

II.1 Две параллельные конструкции

В VR-Sets — цепочка построения:

```

PSet := inductive mk ( $\alpha$  : Type u) (A :  $\alpha \rightarrow$  PSet)
PSet.Equiv x y : взаимная симуляция членства (фундированная рекурсия)
ZFSet := Quotient PSet.setoid
OSet := ZFSet

```

В VR-Sets-ZFA — параллельная конструкция:

```

CoPSetFunctor : PFunctor :=  $\langle$ Type u, id $\rangle$ 
CoPSet := PFunctor.M CoPSetFunctor
CoPSet.Equiv x y : взаимная косимуляция членства (коиндуктивно)
OSetZFA := Quotient CoPSet.cobisim_setoid

```

Структурная параллельность точна: индуктивный PSet соответствует коиндуктивному CoPSet через двойственность финальной коалгебры. PSet использует фундированную рекурсию для равенства; CoPSet использует экстенциональную бисимуляцию. Обе конструкции проходят через квотиент по соответствующей эквивалентности, чтобы получить Lean-тип с правильным экстенциональным равенством.

II.2 Полиномиальный функтор

Полиномиальный функтор, лежащий в основе CoPSet, имеет ту же форму, что и конструктор PSet. В нотации PFunctor:

```

CoPSetFunctor.A = Type u      -- тип формы (индексы детей)
CoPSetFunctor.B = id          -- дети индексируются своей формой

```

Элемент $x : \text{CoPSet}$ раскладывается как (α, A) , где $\alpha : \text{Type } u$ — форма, а $A : \alpha \rightarrow \text{CoPSet}$ — семейство детей. Это разложение доступно через $\text{CoPSet.dest} : \text{CoPSet} \rightarrow \sum \alpha : \text{Type } u, \alpha \rightarrow \text{CoPSet}$, с проекциями CoPSet.shape и CoPSet.children .

Арифметика юниверсов: $\text{CoPSetFunctor} : \text{PFunctor}\{u+1, u\}$, и $\text{PFunctor.M CoPSetFunctor} : \text{Type } (u+1)$. Это точно совпадает с $\text{PSet}\{u\} : \text{Type } (u+1)$, что делает CoPSet drop-in коиндуктивной заменой на том же уровне юниверсов.

II.3 PFunctor.M как субстрат конструкции

PFunctor.M из mathlib (Avigad–Carneiro–Hudon 2019) предоставляет наибольшую неподвижную точку полиномиального функтора как структуру сходящихся последовательностей приближений. Ключевые операции, доступные без дополнительной работы:

```

M.mk      : F (M F) → M F      (конструктор)
M.dest    : M F → F (M F)      (деструктор)

```

`M.corec` : $(X \rightarrow F\ X) \rightarrow X \rightarrow M\ F$ (корекурсор)

`M.bisim` : принцип строгой бисимуляции

`M.corec_unique` : универсальное свойство `M.corec`

`M.corec` и `M.corec_unique` вместе дают то, что называется универсальным свойством финальной коалгебры: каждая коалгебра $(X, X \rightarrow F\ X)$ допускает единственное отображение в $M\ F$, согласованное с коалгебраической структурой. Это в точности содержание AFA Ацеля, когда F — функтор формы множеств. Работа VR-Sets-ZFA состоит в том, чтобы провести это свойство через экстенциональный квотиент и проверить, что оно сохраняется.

II.4 Строгая vs экстенциональная бисимуляция

`M.bisim` — принцип строгой бисимуляции: два M -элемента равны, если связаны отношением R таким, что для каждой R -связанной пары (x, y) формы совпадают как типы, а дети R -связаны индекс-к-индексу. Это требует равенства форм на уровне типов, а не просто эквивалентности кардинальностей.

Теоретико-множественное равенство экстенционально: два множества равны, если имеют одни и те же элементы, независимо от того, как элементы индексируются. Множество $\{\emptyset\}$ может быть представлено как M -элемент формы `Bool` с обоими индексами, отображающимися в $\emptyset$, или как M -элемент формы `Unit` с единственным индексом, отображающимся в $\emptyset$. Эти представления строго бисимилярны в смысле M -элементов (разные формы), так что `M.bisim` их не отождествляет. Они экстенционально бисимилярны — оба представляют синглетон $\{\emptyset\}$ как множество.

Отношение экстенциональной бисимуляции `VR-Sets-ZFA`, `CoPSet.Equiv`, охватывает это:

```
def CoPSet.isBisim (R : CoPSet → CoPSet → Prop) : Prop :=
  ∀ x y, R x y →
    (∀ i : x.shape, ∃ j : y.shape, R (x.children i) (y.children j)) ∧
    (∀ j : y.shape, ∃ i : x.shape, R (x.children i) (y.children j))

def CoPSet.Equiv (x y : CoPSet) : Prop :=
  ∃ R : CoPSet → CoPSet → Prop, CoPSet.isBisim R ∧ R x y
```

Прямые и обратные условия сопоставляют детей по существованию, не по идентичности индексов. Разные по форме представления одного множества `CoPSet.Equiv`-связаны, но не `M.bisim`-связаны. Квотиент по `CoPSet.Equiv` поэтому нетривиален и производит строго другой тип, чем сам `CoPSet`.

II.5 Квотиент OSetZFA

CoPSet.Equiv доказывается рефлексивным, симметричным и транзитивным — стандартные свойства setoid — через экзистенциальные свидетели (тождество, транспозиция, композиция). Свидетель транзитивности — реляционная композиция $R_trans\ a\ c := \exists b, R_1\ a\ b \wedge R_2\ b\ c$, замкнутость по isBisim проверяется цепочкой прямых/обратных симуляционных шагов.

Квотиент далее стандартный:

```
instance CoPSet.instSetoid : Setoid CoPSet := ⟨Equiv, refl, symm, trans⟩  
  
def OSetZFA := Quotient CoPSet.instSetoid
```

Равенство в OSetZFA, обозначаемое $\equiv_ZFA$, тождественно кобисимуляции по построению. Классическая теорема ZFA — что два множества равны тогда и только тогда, когда их графы-картинки бисимилярны — определенациональна в OSetZFA, а не выводится.

* * *

Часть III — AFA как теорема

III.1 Формулировка

Аксиома антифундированности Ацеля в операциональной форме на OSetZFA:

```
theorem AFA_in_OSetZFA :  
   $\forall (V : \text{Type } u) (E : V \rightarrow V \rightarrow \text{Prop}),$   
   $\exists! f : V \rightarrow \text{OSetZFA}, \text{isDecoration } V\ E\ f$ 
```

где isDecoration — экстенциональный предикат декорации:

```
def isDecoration (V : Type u) (E : V → V → Prop) (f : V → OSetZFA) :  
  Prop :=  
   $\forall v : V, \forall x : \text{OSetZFA}, x \in f\ v \leftrightarrow \exists w : V, E\ v\ w \wedge x = f\ w$ 
```

Условие декорации говорит: $f\ v$ имеет своими элементами в точности f -образы E -соседей вершины v . Это стандартное теоретико-множественное содержание AFA на языке декорации графов.

III.2 Существование через M.corec

Декорация строится прямо из графа (V, E) как коалгебра:

```
def graphCoalg (V : Type u) (E : V → V → Prop) (v : V) :=  
  ({w // E v w}, Subtype.val) : PFuncObj CoPSetFuncObj V
```

```
noncomputable def graphCoPSet (V : Type u) (E : V → V → Prop) : V → CoPSet
:=
```

```
  PFunctor.M.corec (graphCoalg V E)
```

```
noncomputable def graphDecoration (V : Type u) (E : V → V → Prop) : V →
  OSetZFA :=
```

```
  fun v => OSetZFA.mk (graphCoPSet V E v)
```

Форма в вершине v — подтип $\{w // E\ v\ w\}$ E -соседей; дети индексируются этими соседями и разворачиваются кореккурсивно. Вычисление деструктора:

```
graphCoPSet_dest : (graphCoPSet V E v).dest =
  ({w // E v w}, fun i => graphCoPSet V E i.val)
```

Доказывается через `rfl`. `PFunctor.M.dest_corec` определенациональна в реализации `mathlib`, поэтому развёртка не требует явных манипуляций леммами.

Существование как половина AFA — `graphDecoration_isDecoration` — следует из развёртки условия декорации через `graphCoPSet_dest` и `CoPSet.mem_mk`.

III.3 Единственность через кобисимуляцию

`M.corec_unique` устанавливает единственность с точностью до строгого M -равенства на уровне M -типа. Это не поднимается прямо к `OSetZFA`, потому что две функции $f, g : V \rightarrow OSetZFA$, удовлетворяющие `isDecoration`, могут иметь `CoPSet`-представители, которые экстенционально бисимилярны, но не строго бисимилярны. Единственность в `OSetZFA` должна устанавливаться на экстенциональном уровне.

Доказательство предъявляет явную кобисимуляцию. Дано f , удовлетворяющее `isDecoration`; классические представители $fRep\ v : CoPSet$ выбираются через `Classical.choose` по `OSetZFA.mk_surjective`. Отношение

```
R c d := ∃ v : V, CoPSet.Equiv c (fRep v) ∧ CoPSet.Equiv d (graphCoPSet V
  E v)
```

показывается бисимуляцией цепочкой: ребёнок c соответствует через `CoPSet.isBisim_Equiv` ребёнку $fRep\ v$, который по `isDecoration` соответствует $fRep\ w$ для некоторого соседа w , что по соответствию классических представителей соответствует ребёнку `graphCoPSet V E v`, индексированному w (через `graphCoPSet_dest`). Обратное направление симметрично.

Затем `CoPSet.bisim_imp_Equiv` даёт `CoPSet.Equiv (fRep v) (graphCoPSet V E v)` для каждого v , откуда `OSetZFA.sound` даёт $f\ v = \text{graphDecoration } V\ E\ v$. Финальная теорема пакетирует существование и единственность:

```
theorem AFA_in_OSetZFA (V : Type u) (E : V → V → Prop) :
  ∃! f : V → OSetZFA, isDecoration V E f :=
  ⟨graphDecoration V E,
    graphDecoration_isDecoration V E,
    fun f hf => graphDecoration_unique V E f hf⟩
```

III.4 AFA как теорема, не аксиома

AFA устанавливается как теорема `VR-Sets-ZFA`, не постулируется как аксиома. Аксиомы доказательства остаются в пределах стандартного потолка `mathlib`: `[propext, Classical.choice, Quot.sound]`. Никаких дополнительных аксиом не вводится.

Это параллельно Теореме III.9 (Выбор) в `VR-Sets`, которая доказывает аксиому выбора как теорему в операциональной онтологии через `Classical.epsilon`. В обоих случаях принцип становится теоремой каркаса, а не постулатом рядом с каркасом. Архитектурная цена несётся подлежащей конструкцией типа (`CoPSet` через `PFunctor.M`, `OSet` через `PSet`); сама теорема коротка, как только субстрат построен правильно.

Методологический выигрыш: `VR-Sets-ZFA` не несёт никаких аксиом, которые потребовала бы классическая `ZFA + AFA`. Фундаментальное расширение достигается без ослабления аксиомного профиля относительно `mathlib`.

* * *

Часть IV — Вложение

IV.1 OSet внутри OSetZFA

Фундированные множества вкладываются в `ZFA`-универсум через структурную рекурсию по `PSet`:

```
def embedPSet : PSet → CoPSet
  | PSet.mk α A => CoPSet.mk α (fun i => embedPSet (A i))
```

Это индуктивное определение (`PSet` индуктивен; $A\ i$ — строгий подтерм $\text{PSet.mk } \alpha\ A$), производящее коиндуктивное значение. `Lean` принимает эту рекурсию через структурное убывание по `PSet`.

Отображение спускается до `OSet` через квотиент посредством `Quotient.lift`, требуя, чтобы `embedPSet` уважал `PSet.Equiv`:

```

noncomputable def embed0Set : OSet → OSetZFA :=
  Quotient.lift (fun p => OSetZFA.mk (embedPSet p))
    (fun _ _ h => OSetZFA.sound (embedPSet_congr h))

```

Лемма согласованности $\text{embedPSet_congr} : \text{PSet.Equiv } x \ y \rightarrow \text{CoPSet.Equiv } (\text{embedPSet } x) \ (\text{embedPSet } y)$ замыкает лифт.

IV.2 Точность вложения

Три теоремы устанавливают точность embed0Set :

```

theorem embedPSet_faithful :
  CoPSet.Equiv (embedPSet x) (embedPSet y) → PSet.Equiv x y

theorem embed0Set_injective : Function.Injective embed0Set

theorem embed0Set_mem (x a : OSet) :
  embed0Set x ∈ embed0Set a ↔ x ∈ a

```

Вместе они говорят: разные фундированные множества отображаются в разные ZFA-множества; членство сохраняется через вложение. Фундированный субуниверсум OSetZFA — образ embed0Set — изоморфен OSet как теория множеств.

IV.3 Асимметричные техники доказательства

Прямое направление согласованности ($\text{PSet.Equiv} \rightarrow \text{CoPSet.Equiv}$) и обратное направление точности ($\text{CoPSet.Equiv} \rightarrow \text{PSet.Equiv}$) требуют разных техник доказательства. Асимметрия фундаментальна для индуктивно-коиндуктивного раздела.

Прямое направление (embedPSet_congr) доказывается чистой бисимуляцией без индукции по PSet . Отношение

$$R \ c \ d := \exists \ x \ y : \text{PSet}, \text{PSet.Equiv } x \ y \wedge c = \text{embedPSet } x \wedge d = \text{embedPSet } y$$

показывается CoPSet -бисимуляцией в одном шаге структурной декомпозиции. $\text{CoPSet.bisim_imp_Equiv}$ замыкает цель коиндуктивно. Коиндуктивный каркас поглощает всю обязанность доказательства.

Обратное направление ($\text{embedPSet_faithful}$) требует структурной индукции по PSet . Дано $\text{CoPSet.Equiv } (\text{embedPSet } (\text{PSet.mk } \alpha \ A)) \ (\text{embedPSet } (\text{PSet.mk } \beta \ B))$; соответствие детей извлекается через $\text{CoPSet.isBisim_Equiv}$, и гипотеза индукции применяется в каждом ребёнке $A \ i$, чтобы получить $\text{PSet.Equiv } (A \ i) \ (B \ j)$ для некоторого j . Фундированность PSet —

его индуктивная структура — является ресурсом доказательства, делающим этот спуск завершающимся.

Схематическое наблюдение: вложение фундированной структуры в коиндуктивный универсум свободно от бисимуляции; извлечение фундированной структуры из коиндуктивной эквивалентности требует фундированности как ресурса доказательства. Асимметрия отслеживает направление индуктивно-коиндуктивного сдвига.

IV.4 ZFC $\not\subseteq$ ZFA — строгое включение

Вложение точно (инъективно, сохраняет членство), но не сюръективно. Атом Куайна — явный свидетель несюръективности:

```
theorem quineAtom_not_in_range_embed0Set :  
  quineAtom  $\notin$  Set.range embed0Set
```

Доказательство: если $\text{embed0Set } x = \text{quineAtom}$, то $x \in x$ (через embed0Set_mem и $\text{quineAtom_self_mem}$), что противоречит ZFSet.mem_wf .

Отображение ZFC  ZFA, следовательно, является строгим вложением, не изоморфизмом. Нефундированные множества — действительно новые объекты в OSetZFA , не переформулировки фундированных множеств.

* * *

Часть V — Демонстрации

V.1 Атом Куайна

Атом Куайна — каноническое самочленное множество, построенное как декорация одновершинного графа с петлёй:

```
noncomputable def quineAtom : OSetZFA :=  
  graphDecoration (fun (_ _ : Unit) => True) ()
```

```
theorem quineAtom_mem_iff (z : OSetZFA) :  
  z  $\in$  quineAtom  $\leftrightarrow$  z = quineAtom
```

```
theorem quineAtom_self_mem : quineAtom  $\in$  quineAtom :=  
  (quineAtom_mem_iff quineAtom).mpr rfl
```

Характеризация членства редуцирует к: единственный член quineAtom — сам quineAtom . Самочленство мгновенно. Каноническое уравнение $q = \{q\}$ тоже выражается прямо:

```
theorem quineAtom_eq_singleton_self :
  quineAtom = OSetZFA.singleton quineAtom
```

Доказывается через OSetZFA.ext из quineAtom_mem_iff и mem_singleton.

V.2 Коллапс бисимуляции

Разные по виду доступные точечные графы могут производить один и тот же элемент OSetZFA. Двухвершинный циклический граф (Bool, ≠) и одновершинный граф с петлёй (Unit, True) оба декорируются в атом Куайна:

```
noncomputable def cycleDecoration : Bool → OSetZFA :=
  graphDecoration (fun x y : Bool => x ≠ y)
```

```
theorem cycleDecoration_eq_quineAtom (b : Bool) :
  cycleDecoration b = quineAtom
```

Доказательство использует CoPSet.bisim_imp_Equiv с отношением

```
R c d := (c = graphCoPSet (≠) false ∨ c = graphCoPSet (≠) true)
  ∧ d = graphCoPSet (fun _ _ : Unit => True) ()
```

Обе вершины двухвершинного цикла и одна вершина с петлёй имеют по одному ребёнку (себя или друг друга соответственно), и после кобисимуляции отождествляются. Граф, который описывает кажущийся двухвершинный цикл, экстенционально является тем же множеством, что и петля.

Это теорема построения, а не его ограничение. Экстенциональная бисимуляция корректно отождествляет эти структуры. Строгая бисимуляция (M.bisim) различала бы их, поскольку типы форм Bool и Unit не равны. Выбор экстенционального квотиента вместо строгой бисимуляции — это в точности то, что делает OSetZFA теорией множеств, а не теорией деревьев.

V.3 Два режима нефундированности

Нефундированность в OSetZFA допускает два различных режима. Атом Куайна иллюстрирует самочленство:

$q \in q \in q \in \dots$ (бесконечная цепь повторением)

Омега-цепь иллюстрирует бесконечный спуск без самочленства:

```
noncomputable def omegaChain : ℕ → OSetZFA :=
  graphDecoration (fun n m : ℕ => m = n + 1)
```

theorem omegaChain_descent (n : ℕ) :

omegaChain (n + 1) ∈ omegaChain n

Здесь цепь $\text{omegaChain } 0 \ni \text{omegaChain } 1 \ni \text{omegaChain } 2 \ni \dots$ не имеет самочленного элемента; каждый шаг спускается к различному множеству. Оба феномена существуют в OSetZFA, ни один не существует в OSet.

V.4 Нефундированность членства OSetZFA

Членство на OSetZFA не является фундированным:

theorem OSetZFA_mem_not_wf :

¬ WellFounded (· ∈ · : OSetZFA → OSetZFA → Prop)

Доказательство: если бы $\cdot \in \cdot$ была фундирована, атом Куайна был бы доступен, и `acc_irrefl` противоречил бы `quineAtom_self_mem`. Это прямо контрастирует с OSet, где `IsWellFounded (· ∈ ·)` — теорема, унаследованная из `mathlib`'s ZFSet через индуктивную структуру PSet.

Два отношения членства, таким образом, имеют категориально различное доказательно-теоретическое поведение. Индукция по членству доступна в OSet; коиндукция доступна в OSetZFA. Вложение `embedOSet` не переносит фундированность, поскольку OSetZFA содержит свидетелей (атом Куайна), которые её нарушают.

* * *

Часть VI — Методологические наблюдения

Наблюдение 1 (M-тип как ZFA-субстрат). `PFunctor.M` из `mathlib`, задуманный как общий конструктор коиндуктивных типов для полиномиальных функторов, прямо использован как коиндуктивный тип формы множеств без модификаций. Полиномиальный функтор $\langle \text{Type } u, \text{id} \rangle$ — форма как тип индекса, дети индексируются формой — точно отражает конструктор PSet. Построение `CoPSet` — одна строка: `def CoPSet := PFunctor.M ⟨Type u, id⟩`. Универсальное свойство `M.corec_unique` потребляется как теорема единственности AFA после экстенционального квотента.

Наблюдение 2 (строгая vs экстенциональная бисимуляция). `PFunctor.M.bisim` — принцип строгой бисимуляции: отождествляет бисимилярные M-элементы, когда они имеют идентичные типы форм и индекс-к-индексу R-связанных детей. Теоретико-множественное равенство экстенционально: типы форм могут различаться, если мультимножества детей совпадают. Это разные отношения на `CoPSet`, и различие нетривиально: два `CoPSet`-элемента, представляющие синглетон $\{\emptyset\}$ с разными типами форм, экстенционально равны, но не строго бисимилярны. Конструкция ZFA через `PFunctor.M`, следовательно, требует экстенционального квотента, а не только встроенного в M-тип сжатия `bisim`.

Наблюдение 3 (AFA как теорема через финальную коалгебру). На экстенциональном квотиенте `OSetZFA` AFA возникает как теорема из `M.corec` (существование) и бисимуляционного аргумента, поднимающего `M.corec_unique` до `OSetZFA` (единственность). Реальная работа находится в построении `CoPSet` и квотиента по кобисимуляции; само доказательство AFA — приблизительно 90 строк. Это параллельно тому, как Теорема III.9 Выбор в `VR-Sets` стала короткой через `Classical.epsilon`: работа смещается от финальной теоремы к подлежащей машинерии.

Наблюдение 4 (кобисимуляция как определенациональное равенство). Классическая теория ZFA выводит результат, что два множества равны тогда и только тогда, когда их графы-картинки бисимильны. В `OSetZFA` это определенационально: $a \equiv_{\text{ZFA}} b$ разворачивается в `Quotient.eq`, которое разворачивается в `CoPSet.Equiv`, которое и есть отношение кобисимуляции. Производная теорема классического подхода становится принципом построения `OSetZFA`.

Наблюдение 5 (асимметричные доказательства вложения). Прямое направление `embedPSet_congr` (`PSet.Equiv` $\rightarrow$ `CoPSet.Equiv`) доказывается чистой бисимуляцией без индукции; отношение, поднимающее `PSet.Equiv` через `embedPSet`, является `CoPSet`-бисимуляцией. Обратное направление (`embedPSet_faithful`) требует структурной индукции по фундированности `PSet` для спуска через коиндуктивную эквивалентность. Вложение фундированной структуры в коиндуктивный универсум свободно от бисимуляции; извлечение фундированной структуры из коиндуктивной эквивалентности требует фундированности как ресурса доказательства. Асимметрия отражает индуктивно-коиндуктивный раздел.

Наблюдение 6 (свободное от аксиом коиндуктивное ядро). Восемь публичных объектов цикла свободны от аксиом: `CoPSetFunctor`, `CoPSet`, `CoPSet.mk`, `CoPSet.corec`, `graphCoalg`, `graphCoPSet`, `embedPSet`, `acc_irrefl`. Классическая машинерия входит только через `CoPSet.dest` (деструктор M-типа, через последовательности приближений, требующие `Classical.choice` для выбора представителя) и принципы бисимуляции. Коиндуктивное содержание конструктивно; классическая машинерия нужна только для его анализа, не для построения.

Наблюдение 7 (определенациональный `dest_corec`). `PFunctor.M.dest_corec` доказывается через `rfl` в `mathlib`. Правило вычисления $(M.corec\ f\ x).dest = \langle \text{shape}, M.corec\ f \circ \text{children} \rangle$, следовательно, определенационально, не требуя `simp`-перезаписи. Пользовательские коиндуктивные конструкции наследуют эту прозрачность: `graphCoPSet_dest` — одна строка `rfl`. Это инженерное преимущество использования `PFunctor.M` как субстрата вместо самодельной коиндуктивной конструкции.

Наблюдение 8 (`change` вместо `simp` для нотационных слоёв). Определенациональные равенства, пересекающие несколько нотационных слоёв (метод инстанса $\rightarrow$ `typeclass-операция` $\rightarrow$ `liftOn2` $\rightarrow$ подлежащая функция), не надёжно сопоставляются через `simp only [...]`. Тактика `change` на уровне ядра, которая принудительно приводит цель к

специфической синтаксической форме перед применением дальнейших тактик, успешна там, где `simp` нет. Этот паттерн появился в `mem_mk` (Этап 4), `graphDecoration_isDecoration` (Этап 5), `embedOSet_mem` (Этап 6), `cycleDecoration_eq_quineAtom` (Этап 7) и `mem_singleton` (Этап 8). Это переносимый инженерный принцип Lean 4 для любой конструкции, наложенной через `Quotient.lift` и `typeclass`-инстансы.

Наблюдение 9 (порядок аргументов в typeclass Membership). Класс `Membership` в Lean 4 имеет `mem :  $\gamma \rightarrow \alpha \rightarrow \text{Prop}$` , где контейнер первый, элемент второй. Нотация $a \in b$ разворачивается в `inst.mem b a`. Пользовательский инстанс `Membership` должен инвертировать естественный порядок аргументов в теле инстанса: `(fun container element => OSetZFA.Mem element container)`. Без этой инверсии последующие леммы вроде `mem_mk` становятся неформулируемыми. Инстанс членства `mathlib's ZFSet` следует тому же паттерну. Это переносимый Lean 4 инженерный урок для определения членства в новых типах, подобных множествам.

Наблюдение 10 (ограничение парсера с doc-комментариями). Атрибут `set_option` в Lean 4 не может появляться между doc-комментарием и последующим объявлением. Парсер после закрытия doc-комментария через `-/` ожидает ключевое слово объявления (`theorem`, `def` и т.д.) или маркер атрибута, а не `set_option`. Обходной путь: подавить предупреждения о неиспользуемых переменных на месте связывания (`fun _ _ hab =>` вместо `fun a _ hab =>`) или поместить `set_option ... in` перед doc-комментарием. Незначительный, но переносимый урок для сложных объявлений.

* * *

Часть VII — Позиция относительно соседних формализаций

VII.1 Работа Полсона в Isabelle/HOL

Полсон (1995–2000-е) формализовал аспекты теории нефундированных множеств в Isabelle/HOL, включая теорию гипермножеств и приложения к семантике языков программирования. Подход использует классические основания HOL напрямую, с гипермножествами, построенными как квотиенты определённых бесконечных деревьев. Работа предшествует современной коалгебраической инфраструктуре в `proof assistant`.

VR-Sets-ZFA отличается в трёх отношениях. Первое: подлежащая теория типов зависимая (Lean 4), а не высшего порядка классическая (Isabelle/HOL); `CoPSet` живёт в `Type (u+1)`, параметризованном уровне юниверса. Второе: субстрат построения — современная коалгебраическая инфраструктура (`PFunctor.M`, `M`-тип конструкции наибольших неподвижных точек), а не прямое древесное кодирование. Третье: VR-Sets-ZFA позиционирована в рамках более широкой программы операциональной онтологии (Цикл VR), а не как самостоятельная теория гипермножеств.

VII.2 Работа Гильтеруда в HoTT

Гильтеруд и соавторы (2018–2025) формализовали нефундированные множества в гомотопической теории типов, используя индуктивно-рекурсивные конструкции и пропозициональный resizing. Установка HoTT позволяет прямое манипулирование деревьями-с-квотиентами как h -множествами, с AFA, выводимым из рассуждений, основанных на унивалентности.

VR-Sets-ZFA отличается доказательно-теоретической силой. Формализации HoTT используют унивалентность и пропозициональный resizing — аксиомы за пределами стандартного потолка Lean. VR-Sets-ZFA оперирует полностью в пределах [propext, Classical.choice, Quot.sound]; никакой унивалентности, никакого resizing. Это делает VR-Sets-ZFA совместимой с классической инфраструктурой mathlib, но утрачивает концептуальную чистоту унивалентных оснований для тождественности множеств.

Подходы дополняют друг друга. Формализации HoTT улавливают категорное содержание (множества как h -множества с бисимуляцией как тождеством) ценой дополнительного аксиоматического обязательства. VR-Sets-ZFA улавливает операциональное содержание (множества как классы кобисимуляции M -элементов) в рамках стандартных оснований Lean.

VII.3 Классический Ацель

Оригинальная трактовка Ацеля (Non-Well-Founded Sets, 1988) представляет AFA как постулат внутри теории множеств, причём универсум множеств расширяется, чтобы включить нефундированные объекты. Картинки (доступные точечные графы) декорируют этот универсум через принцип антифундированности. Бисимуляция выводится как отношение равенства.

VR-Sets-ZFA инвертирует классическое изложение: бисимуляция — принцип построения, AFA — выводимая теорема, универсум определяется через квотиент по бисимуляции. Операциональное содержание то же; порядок построения обратный. Эта инверсия становится возможной благодаря коиндуктивной инфраструктуре: финальная коалгебра функтора формы множеств — это в точности тот универсум, который Ацель постулирует, а её универсальное свойство — это в точности AFA.

VII.4 Позиция в конструктивной математике

VR-Sets-ZFA не является конструктивной математикой в стиле Бишоп. Классическая машинерия (Classical.choice для выбора представителя, propext для нерелевантности доказательств) используется свободно внутри доказательств. Это соответствует общему двухрегистровому аппарату VR: классическое рассуждение разрешено в формальном регистре, причём операциональное содержание извлекается на границах регистров через свидетелей.

Операциональное содержание тем не менее реально. Восемь свободных от аксиом публичных объектов цикла — включая `CoPSet.mk`, `CoPSet.corec`, `embedPSet`, `graphCoalg`, `graphCoPSet` — несут конструктивное содержание прямо. Классическая машинерия входит только при анализе структуры через `M.dest` или принципы бисимуляции. `VR-Sets-ZFA` конструктивна по содержанию, классична по аналитическому методу.

* * *

Список литературы

- Aczel, P. (1988). *Non-Well-Founded Sets*. CSLI Publications.
- Avigad, J., Carneiro, M. & Hudon, S. (2019). Data types as quotients of polynomial functors. *10th International Conference on Interactive Theorem Proving (ITP 2019)*, Leibniz International Proceedings in Informatics.
- Barwise, J. & Moss, L. (1996). *Vicious Circles: On the Mathematics of Non-Wellfounded Phenomena*. CSLI Publications.
- Bishop, E. & Bridges, D. (1985). *Constructive Analysis*. Springer.
- Gylterud, H. R. (2018). From multisets to sets in homotopy type theory. *Journal of Symbolic Logic*, 83(3), 1132–1146.
- Gylterud, H. R., Stenholm, A. & Veltri, N. (2025). Non-wellfounded sets in HoTT/UF. *Logical Methods in Computer Science* (в печати).
- Paulson, L. C. (1999). Final coalgebras as greatest fixed points in ZF set theory. *Mathematical Structures in Computer Science*, 9(5), 545–567.
- Резник, В. (2026a). VR. A formal system. Zenodo. DOI: 10.5281/zenodo.20324391.
- Резник, В. (2026b). VR-Numbers. Zenodo. DOI: 10.5281/zenodo.20352239.
- Резник, В. (2026c). VR-Sets. Zenodo. DOI: 10.5281/zenodo.20354628.
- Резник, В. (2026d). VR-Forms. Zenodo. DOI: 10.5281/zenodo.20355939.
- Резник, В. (2026e). VR-Audit. Zenodo. DOI: 10.5281/zenodo.20364111.
- Резник, В. (2026f). VR-Sets-ZFA Lean 4 formalisation, v1.5-vr-sets-zfa. Zenodo. DOI: 10.5281/zenodo.20368268.

* * *

Благодарности

Работа выполнена с использованием Claude Opus 4.7 (архитектурный обзор) и Claude Sonnet 4.6 (реализация в Lean), Вариант А (интерактивная parent-child архитектура), согласно всем предыдущим Lean-циклам цикла VR.